



מס' קבוצה	אתר	תאריך הגשה
27	https://www.avis.co.il/	26/01/2025
חברי הצוות - מספרי ת.ז.		
322713447	207897489	211693742

פרויקט בסיסי נתונים

רקע כללי על הארגון והאתר

תיאור הארגון AVIS –

AVIS היא חברה בינלאומית להשכרת רכב, שהוקמה בשנת 1946 על ידי וורן אויס בדטרויט, ארה"ב. החברה פועלת כיום ביותר מ-165 מדינות ומפעילה למעלה מ-5,000 סניפים ברחבי העולם.

מטרות עסקיות:

1. לספק פתרונות תחבורה מותאמים אישית ללקוחות פרטיים ועסקיים.
2. להציע שירותי השכרת רכבים לטווח קצר וארוך, כולל שירותי ליסינג ותפעול רכבים.
3. להבטיח זמינות וגמישות מרבית באמצעות צי מגוון של רכבים ושירותי הזמנה מקוונים.

אתגרים עיקריים:

1. **תחרות בענף:** תחום השכרת הרכב כולל מתחרים רבים, מקומיים ובינלאומיים.
2. **עלויות תפעול גבוהות:** תחזוקת צי הרכבים, דלק וביטוח מהווים אתגר מרכזי.
3. **שינויים טכנולוגיים:** פיתוח פלטפורמות הזמנה וניהול מקוונות דורש השקעה מתמשכת.
4. **הסתגלות לצרכים משתנים:** מעבר לרכבים חשמליים והגברת המודעות לאיכות הסביבה.

תפקיד האתר

האתר של AVIS משרת מטרות עסקיות מרכזיות:

- **נגישות גלובלית:** מאפשר ללקוחות להזמין רכבים מכל מקום בעולם.
- **שיפור חוויית המשתמש:** כולל מגוון חיפוש מתקדם, תמיכה בשפות שונות, והתאמה למכשירים ניידים.
- **ניהול לקוחות יעיל:** מעקב אחר הזמנות, תשלומים וניהול מועדוני לקוחות.
- **יעול תפעולי:** חוסך משאבים באמצעות תהליך הזמנה אוטומטי ומספק נתונים עסקיים להנהלת החברה.



התהליך העסקי

תיאור התהליך העסקי – השכרת רכב בחו"ל

התהליך העסקי המרכזי הוא השכרת רכב ללקוחות פרטיים או עסקיים במדינות שונות. התהליך כולל בדיקת זמינות לפי תאריכים ומיקום, חיפוש רכב, הזנת פרטי לקוח ותשלום מקוון. לאחר ההזמנה, הלקוח אוסף את הרכב מהסניף הנבחר, משתמש בו בהתאם לתנאי ההשכרה, ומחזיר אותו בסיום התקופה.

לצד השכרת רכבים, האתר מציע שירותי ליסינג (השכרה לטווח ארוך), מכירת רכבים משומשים ושירותי ניהול צי רכב ללקוחות עסקיים. השכרת הרכב היא הפעילות המרכזית באתר ומהווה את עיקר ההכנסות, בעוד השירותים הנוספים נגזרים ממנה.

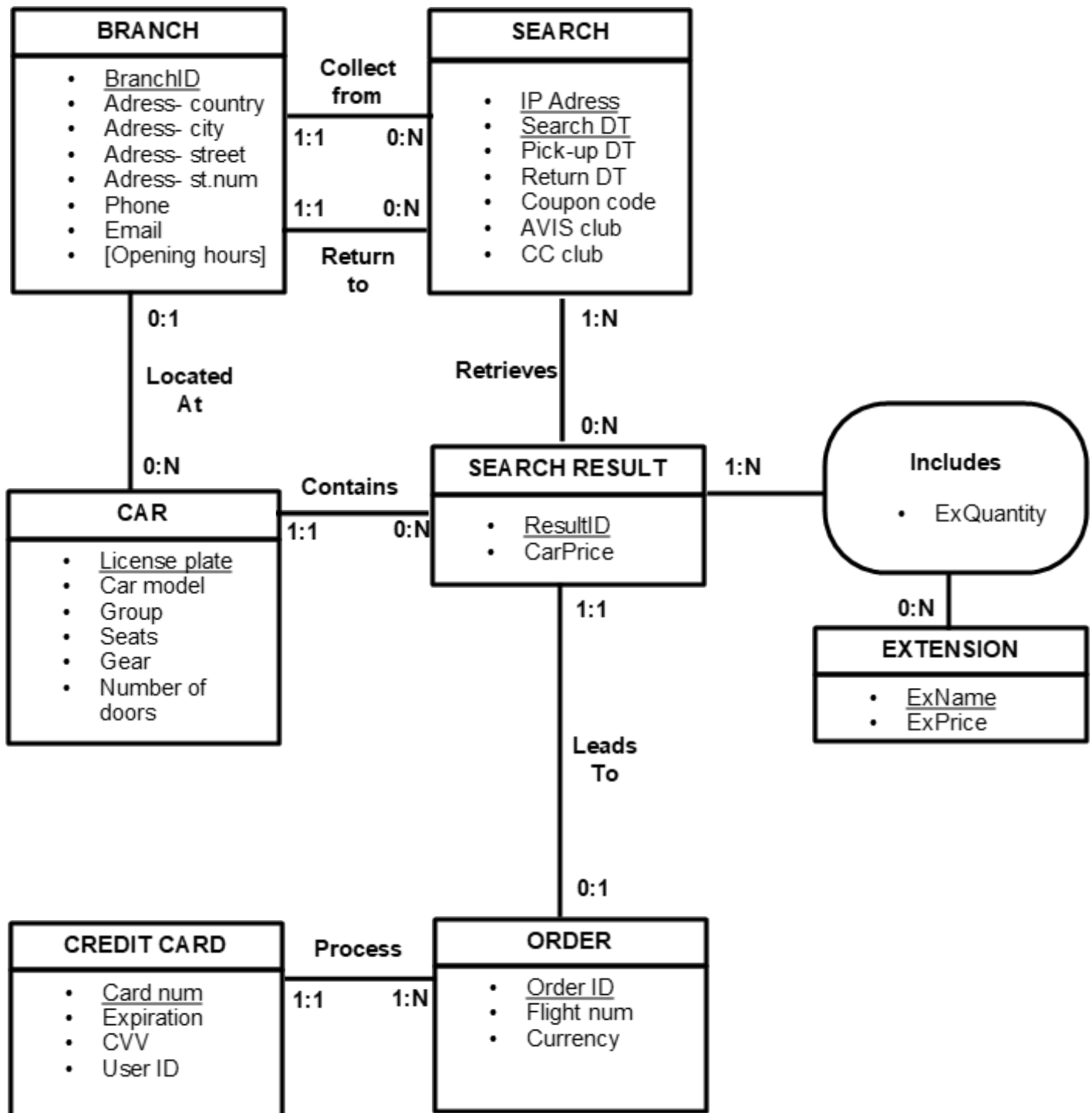
לחברה פעילויות מחוץ לאתר, כמו שירות לקוחות טלפוני, תחזוקת רכבים ושירותים לוגיסטיים בסניפים. התהליך המקוון משתלב בפעילויות אלה, ומייעל את התפעול על ידי חיבור מהיר בין הלקוח לשירותים הפיזיים.

מידע נדרש במהלך התהליך:

מידע חובה: שם מלא, כתובת, מספר טלפון, פרטי תשלום, רישיון נהיגה, תאריכים וסניפי איסוף והחזרה.

מידע אופציונלי: בקשות מיוחדות (כיסא תינוק), GPS, פרטי טיסה, חברות במועדון לקוחות.

לא כל המידע הכרחי לתהליך עצמו אך נדרש לצרכים לוגיסטיים (תיאום זמנים) ושיווקיים (שימור לקוחות ושיפור חוויית השירות).

**ERD:**

**מודל הטבלאות:**

- **ORDERS** (OrderID, FlightNum, Currency, CardNum (CREDIT_CARDS), ResultID (SEARCH_RESULT))
- **CREDIT_CARDS** (CardNum, Expiration, CVV, UserID)
- **BRANCHES** (BranchID, AddressCountry, AddressCity, AddressStreet, AddressStNum, Phone, Email)
- **OPENING-HOURS** (BranchID (BRANCHES), Day, Hour)
- **SEARCHES** (IpAddress, SearchDT, PickupDT, ReturnDT, CouponCode, AvisClub, CcClub, CollectBranchID (BRANCHES), ReturnBranchID (BRANCHES))
- **SEARCH_RESULTS** (ResultID, CarPrice, LicensePlate (CARS))
- **EXTENSIONS** (ExtName, ExtPrice)
- **INCLUSINGS** (ExtName(EXTENSIONS), ResultID(SEARCH_RESULTS), ExtQuantity)
- **CARS** (LicensePlate, CarModel (CARMODELS), Seats, Gear, NumberOfDoors, BranchID(BRANCHES))
- **CARMODELS** (CarModel, Group)
- **RETRIVALS** (ResultID (SEARCH_RESULT)), {IpAddress, SearchDT} (SEARCHES))



שאלות

שתי שאלות SELECT ללא קינון

1. בשאלתה זו נציג את 5 קבוצות הרכבים (Vehicle Groups) המניבות את סך ההכנסות הגבוה ביותר מבין הקבוצות שאינן שייכות לקטגוריות A ו-B. בנוסף, נציג את מספר ההזמנות הכולל, סך ההכנסות, ומחיר ההשכרה הממוצע עבור כל קבוצה. הרעיון העסקי הינו לסייע לחברה בקבלת החלטות אסטרטגיות על הרחבת צי הרכבים בקבוצות המניבות הכנסה גבוהה. כמו כן, לסייע בתכנון מבצעי שיווק ממוקדים להגברת הביקוש בקבוצות נוספות או שימור הביקוש בקבוצות הרווחיות, באמצעות סיפוק מידע על הפופולריות של קבוצות רכבים מסוימות.

```
SELECT top 5
  CM.[Group] AS VehicleGroup,
  COUNT(O.OrderID) AS TotalOrders,
  SUM(SR.CarPrice) AS TotalRevenue,
  AVG(SR.CarPrice) AS AveragePrice
FROM
  ORDERS O
JOIN
  SEARCH_RESULTS SR ON O.ResultID = SR.ResultID
JOIN
  CARS C ON SR.LicensePlate = C.LicensePlate
JOIN
  CARMODELS CM ON C.CarModel = CM.CarModel
WHERE
  CM.[Group] NOT IN ('A', 'B')
GROUP BY
  CM.[Group]
HAVING
  COUNT(O.OrderID) > 0
ORDER BY
  TotalRevenue DESC;
```

הפלט שהתקבל:

	VehicleGroup	TotalOrders	TotalRevenue	AveragePrice
1	F	74	25792.17	348.5428
2	H	27	14693.10	544.1888
3	C	72	12715.12	176.5988
4	G	14	6433.94	459.5671
5	E	16	4325.80	270.3625



2. בשאלתה זו נציג את 5 התוספות (Extensions) המובילות מבחינת סך ההכנסות בשנת 2023 ו-2024. עבור כל תוספת, יוצגו הכמות הכוללת שנמכרה (TotalQuantity) וסך ההכנסות (TotalRevenue). התוצאות מסודרות לפי סך ההכנסות בסדר יורד, ואם יש שוויון, תינתן עדיפות לתוספת עם כמות המכירות הגדולה יותר.

ניתוח ההכנסות והשימוש בתוספות פופולריות בשנתיים האחרונות מספק תובנות קריטיות לתכנון אסטרטגי. על ידי הבנה אילו תוספות מניבות את סך ההכנסות הגבוה ביותר, ניתן לזהות מגמות בביקוש, לשפר את ניהול המלאי, ולהתאים את מבני התמחור בהתאם. בנוסף, ניתוח הפערים בין כמות המכירות להכנסות יכול לשמש כאינדיקציה לאפקטיביות של מבצעי שיווק ומחירים, ובכך לסייע ביצירת קמפיינים ממוקדים או בפיתוח חבילות מותאמות אישית ללקוחות.

```
SELECT TOP 5
    E.ExtName,
    SUM(IR.ExtQuantity) AS TotalQuantity,
    SUM(E.ExtPrice * IR.ExtQuantity) AS TotalRevenue
FROM
    INCLUSIONS IR
JOIN
    EXTENSIONS E ON IR.ExtName = E.ExtName
JOIN
    RETRIEVALS R ON IR.ResultID = R.ResultID
JOIN
    SEARCHES S ON R.IPAddress = S.IPAddress AND R.SearchDT = S.SearchDT
WHERE
    YEAR(S.SearchDT) IN (2023, 2024)
GROUP BY
    E.ExtName
ORDER BY
    TotalRevenue DESC, TotalQuantity DESC;
```

הפלט שהתקבל:

	ExtName	TotalQuantity	TotalRevenue
1	GPS	92	5969.5764
2	Additional Driver	72	1137.6576
3	Child Seat	28	581.448
4	Snow Chains	24	556.9224
5	AVIS Tow Sec...	11	505.0914



שתי שאלות SELECT מקוננות

1. בשאלתה זו נציג את אחוז הרווחים הכוללים שהתקבלו מלקוחות שהם חברי מועדון AvisClub ו-CCClub מתוך סך כל הרווחים שהתקבלו מהזמנות שבוצעו בפועל (מתוך טבלת ORDERS). התוצאה מחושבת בנפרד עבור כל אחד מהמועדונים, כך שניתן להבחין בתרומת ההכנסות של כל מועדון באופן עצמאי. הרעיון העסקי הוא להבין את התרומה של כל מועדון לרווחי החברה. במידה והתוצאות מראות שחברי אחד המועדונים אחראים לחלק משמעותי מהרווחים, ניתן לשקול להרחיב את הפעילות הקשורה למועדון זה, להציע הטבות נוספות או למקד קמפיינים שיווקיים שימשכו לקוחות נוספים להצטרף למועדון. כמו כן, אם נמצא שמועדון מסוים תורם פחות, ייתכן שנדרשת בחינה מחדש של מדיניות המועדון והטבותיו כדי לשפר את הביצועים.

```
SELECT
  CAST(
    (SELECT
      SUM(SR1.CarPrice)
    FROM
      SEARCH_RESULTS SR1
    WHERE
      SR1.ResultID IN (
        SELECT
          R1.ResultID
        FROM
          RETRIVALS R1
        INNER JOIN SEARCHES S1
        ON R1.IpAddress = S1.IpAddress AND R1.SearchDT = S1.SearchDT
        WHERE
          S1.AvisClub IS NOT NULL
          AND R1.ResultID IN (SELECT ResultID FROM ORDERS)
      )
    ) * 100.0 /
    (SELECT
      SUM(SR2.CarPrice)
    FROM
      SEARCH_RESULTS SR2
    WHERE
      SR2.ResultID IN (SELECT ResultID FROM ORDERS)
    ) AS INT
  ) AS PercentageRevenueFromAvisClub,
  CAST(
    (SELECT
      SUM(SR1.CarPrice)
    FROM
      SEARCH_RESULTS SR1
    WHERE
      SR1.ResultID IN (
        SELECT
```



```

        R1.ResultID
FROM
    RETRIVALS R1
INNER JOIN SEARCHES S1
ON R1.IpAddress = S1.IpAddress AND R1.SearchDT = S1.SearchDT
WHERE
    S1.CcClub IS NOT NULL
    AND R1.ResultID IN (SELECT ResultID FROM ORDERS)
    )
) * 100.0 /
(SELECT
    SUM(SR2.CarPrice)
FROM
    SEARCH_RESULTS SR2
WHERE
    SR2.ResultID IN (SELECT ResultID FROM ORDERS)
) AS INT
) AS PercentageRevenueFromCcClub;

```

הפלט שהתקבל:

	PercentageRevenueFromAvisClub	PercentageRevenueFromCcClub
1	18	18



2. בשאלתה זו נחשב את 10 הסניפים בעלי כמות ההשכרות הנמוכה מהממוצע הכללי של כמות ההשכרות בכלל הסניפים (פרט לסניפים חדשים). התוצאה מציגה את מזהה הסניף (BranchID), את עיר הסניף (AdressCity) ואת סך ההשכרות שביצע כל סניף (RentalsCount). הרעיון העסקי שמאחורי השאלתה הוא לזהות סניפים חלשים מבחינת ביצועים ולספק למנהלי השיווק תובנות שיסייעו במיקוד מאמצי השיווק והקידום בסניפים הללו. המידע יכול לשמש לפיתוח אסטרטגיות שיווקיות מותאמות, כגון מבצעים ייעודיים או שיתופי פעולה מקומיים, במטרה להגדיל את מספר ההשכרות ולשפר את ביצועי הסניפים החלשים.

```
SELECT TOP 10
    B.BranchID,
    B.AdressCity,
    COUNT(*) AS RentalsCount
FROM
    BRANCHES B
JOIN
    CARS C ON B.BranchID = C.BranchID
JOIN
    SEARCH_RESULTS SR ON C.LicensePlate = SR.LicensePlate
JOIN
    ORDERS O ON SR.ResultID = O.ResultID
GROUP BY
    B.BranchID,
    B.AdressCity
HAVING
    COUNT(*) < (
        SELECT AVG(RentalsCount)
        FROM (
            SELECT
                Car.BranchID,
                COUNT(*) AS RentalsCount
            FROM
                ORDERS O
            JOIN
                SEARCH_RESULTS SR ON O.ResultID = SR.ResultID
            JOIN
                CARS Car ON SR.LicensePlate = Car.LicensePlate
            GROUP BY
                Car.BranchID
        ) AS AvgRentals
    )
ORDER BY RentalsCount ASC;
```



הפלט שהתקבל:

	BranchID	AdressCity	RentalsCount
1	1	Rome	1
2	74	Tel Aviv	1
3	78	Paris	1
4	100	Paris	1
5	10	Kyoto	2
6	3	New York	2
7	31	Rome	2
8	67	New York	2
9	81	Tel Aviv	2
10	88	New York	2



שאלות עסקיות המשלבות Window Functions

1. בשאלתה זו ניתוח פעילות החיפושים וההזמנות בסניפי החברה מתבצע במטרה לזהות את רמת היעילות של כל סניף בהמרת חיפושים להזמנות. השאלתה מחשבת את אחוזי ההמרה לכל סניף על ידי השוואת מספר תוצאות החיפוש שהובילו להזמנות בפועל, לצד סיכום מספר ההזמנות הכולל לכל מדינה ודירוג יחסי של הסניפים בתוך כל מדינה על פי אחוזי ההמרה. התוצאות מספקות תמונה ברורה על הביצועים של כל סניף, גם ברמה מקומית וגם בהשוואה למדדים אזוריים.

הרעיון העסקי של השאלתה הוא לספק תובנות מקיפות על יעילות הסניפים, זיהוי של סניפים שמצטיינים בהמרות בתוך כל מדינה, והבנת הפערים בסניפים שבהם יש הרבה חיפושים אך מעט הזמנות. מידע זה יכול לסייע בשיפור אסטרטגיות שיווקיות, התאמת השירותים לכל סניף, תיקון תקלות בתהליך ההזמנה (אם קיימות ומובילות לאחוזי המרה נמוכים), בהתאם לרמת הביצועים היחסית והכוללת של כל סניף.

```

SELECT
    B.BranchID,
    B.AddressCity,
    B.AddressCountry,
    COUNT(SR.ResultID) AS TotalSearchResults,
    COUNT(O.OrderID) AS TotalOrders,
    CASE
        WHEN COUNT(SR.ResultID) = 0 THEN 0
        ELSE CAST(COUNT(O.OrderID) AS FLOAT) / COUNT(SR.ResultID) * 100
    END AS ConversionRate,
    SUM(COUNT(O.OrderID)) OVER (PARTITION BY B.AddressCountry) AS
TotalOrdersPerCountry,
    RANK() OVER (PARTITION BY B.AddressCountry ORDER BY
        CASE
            WHEN COUNT(SR.ResultID) = 0 THEN 0
            ELSE CAST(COUNT(O.OrderID) AS FLOAT) / COUNT(SR.ResultID) * 100
        END DESC
    ) AS ConversionRankPerCountry
FROM
    BRANCHES B
LEFT JOIN
    SEARCHES S ON B.BranchID = S.CollectBranchID
LEFT JOIN
    RETRIVALS R ON S.IPAddress = R.IPAddress AND S.SearchDT = R.SearchDT
LEFT JOIN
    SEARCH_RESULTS SR ON R.ResultID = SR.ResultID
LEFT JOIN
    ORDERS O ON SR.ResultID = O.ResultID
GROUP BY
    B.BranchID,
    B.AddressCity,
    B.AddressCountry
ORDER BY

```



ConversionRate DESC, ConversionRankPerCountry;

הפלט שהתקבל:

	BranchID	AdressCity	AdressCountry	TotalSearchResults	TotalOrders	ConversionRate	TotalOrdersPerCountry	ConversionRankPerCountry
1	44	Paris	France	2	2	100	17	1
2	17	Paris	France	3	3	100	17	1
3	39	Tel Aviv	Israel	3	2	66.666666666...	10	1
4	79	Rome	Italy	5	3	60	22	1
5	67	New York	USA	2	1	50	13	1
6	10	Kyoto	Japan	4	2	50	2	1
7	62	Rome	Italy	2	1	50	22	2
8	51	Paris	France	7	3	42.857142857...	17	3
9	18	New York	USA	6	2	33.333333333...	13	2
10	28	Houston	USA	12	4	33.333333333...	13	2
11	85	Rome	Italy	3	1	33.333333333...	22	3



2. בשאלתה זו מבוצע ניתוח של הזמנות רכבים לפי עונות השנה ומדינות שונות, תוך זיהוי קבוצות רכבים שנמצאות במיקומים גיאוגרפיים ספציפיים (אירופה). השאלתה מחלקת את הזמנות הרכבים לארבעה רבעונים בעונת השנה, ומבצעת דירוג על פי סך ההכנסות והרווח מההזמנות לכל קבוצת רכבים, עבור כל עונה. ההיגיון העסקי של השאלתה הינו לזהות את קבוצות הרכבים הפופולריות ביותר בעונות השנה השונות, ולספק דירוגים של ההכנסות ורמות הפעילות עבור כל עונה. המידע הזה מסייע בהבנה אילו רכבים מבוקשים יותר בעונות מסוימות, ובכך מאפשר להקצות משאבים בצורה אופטימלית ולפתח אסטרטגיות שיווקיות ממוקדות לכל עונה ומדינה.

SELECT

CM.[Group] AS VehicleGroup,

CASE

WHEN DATEPART(QUARTER, S.PickUpDT) = 1 THEN 'Winter'

WHEN DATEPART(QUARTER, S.PickUpDT) = 2 THEN 'Spring'

WHEN DATEPART(QUARTER, S.PickUpDT) = 3 THEN 'Summer'

WHEN DATEPART(QUARTER, S.PickUpDT) = 4 THEN 'Fall'

END AS SeasonName,

COUNT(O.OrderID) AS TotalOrders,

SUM(SR.CarPrice) AS TotalRevenue,

RANK() OVER (PARTITION BY

CASE

WHEN DATEPART(QUARTER, S.PickUpDT) = 1 THEN 'Winter'

WHEN DATEPART(QUARTER, S.PickUpDT) = 2 THEN 'Spring'

WHEN DATEPART(QUARTER, S.PickUpDT) = 3 THEN 'Summer'

WHEN DATEPART(QUARTER, S.PickUpDT) = 4 THEN 'Fall'

END

ORDER BY SUM(SR.CarPrice) DESC) AS RevenueRank,

NTILE(4) OVER (PARTITION BY

CASE

WHEN DATEPART(QUARTER, S.PickUpDT) = 1 THEN 'Winter'

WHEN DATEPART(QUARTER, S.PickUpDT) = 2 THEN 'Spring'

WHEN DATEPART(QUARTER, S.PickUpDT) = 3 THEN 'Summer'

WHEN DATEPART(QUARTER, S.PickUpDT) = 4 THEN 'Fall'

END

ORDER BY COUNT(O.OrderID) DESC) AS OrderQuartile

FROM

ORDERS O

JOIN

SEARCH_RESULTS SR ON O.ResultID = SR.ResultID

JOIN

RETRIVALS R ON SR.ResultID = R.ResultID

JOIN

SEARCHES S ON S.IPAddress = R.IPAddress

AND S.SearchDT = R.SearchDT

JOIN

CARS C ON SR.LicensePlate = C.LicensePlate



```

JOIN
  CARMODELS CM ON C.CarModel = CM.CarModel
JOIN
  BRANCHES BR ON C.BranchID = BR.BranchID
WHERE
  BR.AddressCountry IN ('France', 'Germany', 'Spain', 'Italy', 'United Kingdom')
GROUP BY
  CM.[Group],
  CASE
    WHEN DATEPART(QUARTER, S.PickUpDT) = 1 THEN 'Winter'
    WHEN DATEPART(QUARTER, S.PickUpDT) = 2 THEN 'Spring'
    WHEN DATEPART(QUARTER, S.PickUpDT) = 3 THEN 'Summer'
    WHEN DATEPART(QUARTER, S.PickUpDT) = 4 THEN 'Fall'
  END
ORDER BY
  SeasonName, RevenueRank;

```

הפלט שהתקבל:

	VehicleGroup	SeasonName	TotalOrders	TotalRevenue	RevenueRank	OrderQuartile
1	F	Fall	2	671.46	1	1
2	H	Fall	1	507.02	2	3
3	C	Fall	3	499.38	3	1
4	G	Fall	1	429.41	4	4
5	A	Fall	2	187.25	5	2
6	H	Spring	4	2178.12	1	1
7	G	Spring	2	943.96	2	2
8	C	Spring	3	533.08	3	2
9	A	Spring	6	413.21	4	1
10	F	Spring	1	334.96	5	3
11	B	Spring	1	120.26	6	4
12	F	Summer	3	1031.25	1	1
13	C	Summer	3	524.57	2	2
14	G	Summer	1	410.73	3	3
15	A	Summer	1	90.86	4	4
16	H	Winter	2	1055.96	1	2
17	F	Winter	3	984.01	2	1
18	C	Winter	2	339.84	3	1
19	D	Winter	1	229.28	4	3
20	A	Winter	1	90.86	5	4

**שאלתה מקוננת תוך שימוש ב-CTE (פסקת WITH) :**

בשאלתה זו ניתוח פעילות החיפושים וההזמנות העתידיות באתר מתבצע במטרה לזהות את הצרכים הצפויים של סניפי החברה. השאלתה מספקת מידע על מספר החיפושים וההזמנות לכל סניף, תוך חישוב כמות הרכבים שיש להקצות לכל סניף על בסיס הביקוש הצפוי, עם תוספת של 20% לעמידה בביקושים עתידיים. בנוסף, השאלתה מדרגת את הסניפים על פי כמות החיפושים העתידיים.

הרעיון העסקי של השאלתה הוא לסייע בקבלת החלטות אסטרטגיות בנוגע להקצאת משאבים לכל סניף, כמו רכבים זמינים, כוח אדם ושעות פעילות, על בסיס רמת הביקוש הצפויה. זיהוי סניפים עם פעילות גבוהה מאפשר להיערך מבעוד מועד לעומס צפוי, בעוד שסניפים עם פעילות נמוכה עשויים להזדקק למיקוד שיווקי או מבצעים ייעודיים. מידע זה תומך בתכנון יעיל ואופטימלי של פעילות החברה.

WITH

```
SearchActivity AS (
    SELECT
        S.CollectBranchID AS BranchID,
        COUNT(*) AS TotalFutureSearches
    FROM
        SEARCHES S
    WHERE
        S.PickUpDT >= DATEADD(MONTH, 1, '2024-01-01')
    GROUP BY
        S.CollectBranchID
),
OrderActivity AS (
    SELECT
        B.BranchID,
        COUNT(O.OrderID) AS TotalFutureOrders
    FROM
        BRANCHES B
    LEFT JOIN
        SEARCHES S ON B.BranchID = S.CollectBranchID
    LEFT JOIN
        RETRIVALS R ON S.IPAddress = R.IPAddress AND S.SearchDT = R.SearchDT
    LEFT JOIN
        SEARCH_RESULTS SR ON R.ResultID = SR.ResultID
    LEFT JOIN
        ORDERS O ON SR.ResultID = O.ResultID
    WHERE
        S.PickUpDT >= DATEADD(MONTH, 1, '2024-01-01')
    GROUP BY
        B.BranchID
),
RankedBranches AS (
    SELECT
        SA.BranchID,
        ISNULL(SA.TotalFutureSearches, 0) AS TotalFutureSearches,
        ISNULL(OA.TotalFutureOrders, 0) AS TotalFutureOrders,
```



```

        NTILE(5) OVER (ORDER BY ISNULL(SA.TotalFutureSearches, 0) DESC) AS
PopularityRank
FROM
    SearchActivity SA
LEFT JOIN
    OrderActivity OA ON SA.BranchID = OA.BranchID
UNION
SELECT
    OA.BranchID,
    ISNULL(SA.TotalFutureSearches, 0) AS TotalFutureSearches,
    ISNULL(OA.TotalFutureOrders, 0) AS TotalFutureOrders,
    NTILE(5) OVER (ORDER BY ISNULL(SA.TotalFutureSearches, 0) DESC) AS
PopularityRank
FROM
    OrderActivity OA
LEFT JOIN
    SearchActivity SA ON OA.BranchID = SA.BranchID
)
SELECT
    RB.BranchID,
    B.AddressCity,
    B.AddressCountry,
    RB.TotalFutureSearches,
    RB.TotalFutureOrders,
    CEILING((RB.TotalFutureSearches + RB.TotalFutureOrders) * 1.2) AS RequiredVehicles,
    RB.PopularityRank
FROM
    RankedBranches RB
JOIN
    BRANCHES B ON RB.BranchID = B.BranchID
ORDER BY
    RB.PopularityRank ASC, RB.TotalFutureSearches DESC;

```

הפלט שהתקבל:

	BranchID	AdressCity	AdressCountry	TotalFutureSearches	TotalFutureOrders	RequiredVehicles	PopularityRank
1	73	New York	USA	6	0	8	1
2	49	New York	USA	5	0	6	1
3	70	Paris	France	5	1	8	1
4	75	Rome	Italy	5	1	8	1
5	9	Tel Aviv	Israel	4	0	5	1
6	36	Houston	USA	4	1	6	1
7	38	Paris	France	4	1	6	1
8	43	Rome	Italy	4	1	6	1
9	1	Rome	Italy	3	1	5	1
10	3	New York	USA	3	0	4	1
11	12	Paris	France	3	0	4	1
12	16	Rome	Italy	3	0	4	1



יישומי כלים מתקדמים

View

ה-View UnifiedBranchData מספק ניתוח מרכזי של ביצועי הסניפים, כולל מדדים כמו סך החיפוש, ההזמנות, אחוזי ההמרה, וההכנסות בכל סניף. מידע זה מאפשר למנהלי החברה לזהות סניפים בעלי ביקוש גבוה, לייעל את התפעול באזורים עם אחוזי המרה נמוכים, ולבנות אסטרטגיות שיווקיות ממוקדות בהתאם לפוטנציאל הרווחי של כל סניף. השימוש ב-View חוסך כתיבת שאילתות חוזרות ומאפשר בסיס נתונים אחיד להצגת דוחות ודשבורדים.

```
CREATE VIEW UnifiedBranchData AS
SELECT
    B.BranchID,
    B.AddressCity,
    B.AddressCountry,
    COUNT(SR.ResultID) AS TotalSearchResults,
    COUNT(O.OrderID) AS TotalOrders,
    CASE
        WHEN COUNT(SR.ResultID) = 0 THEN 0
        ELSE CAST(COUNT(O.OrderID) AS FLOAT) / COUNT(SR.ResultID) * 100
    END AS ConversionRate,
    SUM(SR.CarPrice) AS TotalRevenue
FROM
    BRANCHES B
LEFT JOIN
    SEARCHES S ON B.BranchID = S.CollectBranchID
LEFT JOIN
    RETRIVALS R ON S.IpAddress = R.IpAddress AND S.SearchDT = R.SearchDT
LEFT JOIN
    SEARCH_RESULTS SR ON R.ResultID = SR.ResultID
LEFT JOIN
    ORDERS O ON SR.ResultID = O.ResultID
GROUP BY
    B.BranchID,
    B.AddressCity,
    B.AddressCountry;
```

פונקציות (Functions)

1. הפונקציה CalculateTotalOrderPriceWithDiscount נועדה לחשב את מחיר ההזמנה הכולל, כולל תוספות והנחות, בצורה מרכזית, יעילה וגמישה. היא מפשטת את התחזוקה, מונעת שכפול קוד, ותומכת בשינויים עסקיים עתידיים, תוך שיפור ביצועים וניהול תמחור אחיד.

```
CREATE FUNCTION CalculateTotalOrderPriceWithDiscount (@OrderID INT)
RETURNS MONEY
AS
BEGIN
    DECLARE @TotalPrice MONEY;
    DECLARE @DiscountPercentage TINYINT;

    SELECT
        @DiscountPercentage = ISNULL(CC.Discount, 0)
    FROM
        ORDERS O
    JOIN SEARCH_RESULTS SR ON O.ResultID = SR.ResultID
    JOIN RETRIVALS R ON SR.ResultID = R.ResultID
    JOIN SEARCHES S ON R.IPAddress = S.IPAddress AND R.SearchDT = S.SearchDT
    LEFT JOIN COUPON_CODES CC ON S.CouponCode = CC.CouponCode
    WHERE
        O.OrderID = @OrderID;

    SELECT
        @TotalPrice =
        (
            (SELECT
                SR.CarPrice
            FROM
                ORDERS O
            JOIN SEARCH_RESULTS SR ON O.ResultID = SR.ResultID
            WHERE O.OrderID = @OrderID)
            +
            (SELECT
                ISNULL(SUM(E.ExtPrice * I.ExtQuantity), 0)
            FROM
                ORDERS O
            JOIN SEARCH_RESULTS SR ON O.ResultID = SR.ResultID
            JOIN RETRIVALS R ON SR.ResultID = R.ResultID
            JOIN INCLUSIONS I ON SR.ResultID = I.ResultID
            JOIN EXTENSIONS E ON I.ExtName = E.ExtName
            WHERE O.OrderID = @OrderID)
        );

    IF @DiscountPercentage > 0
        SET @TotalPrice = @TotalPrice * (1 - CAST(@DiscountPercentage AS FLOAT) / 100);
```



```
RETURN @TotalPrice;  
END;
```

```
SELECT dbo.CalculateTotalOrderPriceWithDiscount(123) AS TotalOrderPriceWithDiscount;
```

הפלט שהתקבל:

	TotalOrderPriceWithDiscount
1	116.70



2. הפונקציה GetBranchesDataByCountry מספקת דרך יעילה לשלוף מידע מפורט על סניפים במדינה מסוימת, כולל נתוני המרה, הכנסות ומספר הזמנות, על בסיס ה-VIEW- UnifiedBranchData שמרכז נתונים רלוונטיים מכל הטבלאות הקשורות. השימוש ב-VIEW- מאפשר לפונקציה לעבוד על נתונים מעובדים מראש, ובכך לשפר ביצועים, לצמצם כפילות קוד, ולהקל על תחזוקה. הפונקציה חיונית לתמיכה בניתוחי ביצועים לפי אזורים גיאוגרפיים ולתכנון אסטרטגי של משאבים ושיווק.

```
CREATE FUNCTION GetBranchesDataByCountry (@Country VARCHAR(30))
RETURNS TABLE
AS
RETURN
(
    SELECT
        BranchID,
        AdressCity,
        TotalOrders,
        ConversionRate,
        TotalRevenue
    FROM
        UnifiedBranchData
    WHERE
        AdressCountry = @Country
);
```

```
SELECT *
FROM GetBranchesDataByCountry('USA')
ORDER BY ConversionRate DESC;
```

הפלט שהתקבל:

	BranchID	AdressCity	TotalOrders	ConversionRate	TotalRevenue
1	67	New York	1	50	670.28
2	18	New York	2	33.33333333333333	1282.90
3	28	Houston	4	33.33333333333333	2141.70
4	52	Houston	1	25	1173.82
5	36	Houston	2	16.66666666666667	2577.20
6	7	New York	1	16.66666666666667	1621.71
7	58	New York	1	14.2857142857143	1934.83
8	91	New York	1	14.2857142857143	2190.29



Trigger

הטריגר trg_UpdateCarBranchAssignment נועד לנהל באופן אוטומטי את שיוך הרכבים לסניפים בהתאם לפעולות על טבלת ההזמנות (ORDERS). בעת הוספת הזמנה חדשה, הטריגר מסמן את הרכב שהוזמן כלא משויך לסניף, מה שמבטיח שהוא אינו זמין להזמנות נוספות. לעומת זאת, בעת מחיקת הזמנה (למשל במקרה של ביטול), הטריגר מחזיר את הרכב לסניף המקורי ממנו נאסף. פעולה זו מבטיחה ניהול חכם ואוטומטי של זמינות הרכבים, מפחיתה את הצורך בעדכון ידני, ותורמת לייעול תהליך ניהול המלאי בסניפי החברה.

```
CREATE TRIGGER trg_UpdateCarBranchAssignment
ON ORDERS
AFTER INSERT, DELETE
AS
BEGIN
    IF EXISTS (SELECT 1 FROM INSERTED)
    BEGIN
        UPDATE C
        SET C.BranchID = NULL
        FROM CARS C
        JOIN SEARCH_RESULTS SR ON C.LicensePlate = SR.LicensePlate
        JOIN INSERTED I ON SR.ResultID = I.ResultID;
    END

    IF EXISTS (SELECT 1 FROM DELETED)
    BEGIN
        UPDATE C
        SET C.BranchID = (SELECT B.BranchID
                        FROM SEARCHES S
                        JOIN RETRIVALS R ON S.IpAddress = R.IpAddress AND S.SearchDT =
R.SearchDT
                        JOIN SEARCH_RESULTS SR ON R.ResultID = SR.ResultID
                        JOIN DELETED D ON SR.ResultID = D.ResultID
                        JOIN BRANCHES B ON S.CollectBranchID = B.BranchID
                        WHERE D.ResultID = SR.ResultID)
        FROM CARS C
        JOIN SEARCH_RESULTS SR ON C.LicensePlate = SR.LicensePlate
        JOIN DELETED D ON SR.ResultID = D.ResultID;
    END

    SELECT CASE
        WHEN COUNT(*) > 0 THEN 'Yes'
        ELSE 'No'
    END AS IsInBranch28
FROM CARS
WHERE LicensePlate = 'LBUJDE' AND BranchID = 28;
```



בדיקה לטריגר:

```
INSERT INTO ORDERS (OrderID, FlightNum, Currency, CardNum, ResultID)
VALUES (9999, 'FL123', 'USD', '2136444691581134', 1);
```

לפני הפעלת הטריגר

The screenshot shows a SQL IDE with two queries. The first query selects all columns from the CARS table where the license plate is 'LBUJDE'. The second query is a CASE statement that checks if there is at least one car with license plate 'LBUJDE' in branch 28, returning 'Yes' or 'No'.

```
-- בודקת האם הרכב נמצא בסניף
SELECT CASE
    WHEN COUNT(*) > 0 THEN 'Yes'
    ELSE 'No'
END AS IsInBranch28
FROM CARS
WHERE LicensePlate = 'LBUJDE' AND BranchID = 28;
```

Below the queries, there are two result tables. The first table shows the details of the car with license plate 'LBUJDE'. The second table shows the result of the CASE statement, which is 'Yes'.

	LicensePlate	CarModel	Seats	Gear	NumberOfDoors	BranchID
1	LBUJDE	Toyota 72	5	Automatic	2	28

	IsInBranch28
1	Yes



אחרי הפעלת הטריגר (פעולת INSERT)

```

SELECT *
FROM CARS
WHERE LicensePlate = 'LBUJDE';

-- בודקת האם הרכב נמצא בסניף
SELECT CASE
    WHEN COUNT(*) > 0 THEN 'Yes'
    ELSE 'No'
    END AS IsInBranch28
FROM CARS
WHERE LicensePlate = 'LBUJDE' AND BranchID = 28;

```

100 %

Results Messages

	LicensePlate	CarModel	Seats	Gear	NumberOfDoors	BranchID
1	LBUJDE	Toyota 72	5	Automatic	2	NULL

	IsInBranch28
1	No



פרוצדורה שמורה (Stored Procedure)

הפרוצדורה `AdjustBranchResourcesByFuturePickUp` נועדה לנהל בצורה אוטומטית ויעילה את מלאי הרכבים בין סניפים על בסיס תחזיות ביקוש עתידיות. היא מזהה סניפים עם מחסור צפוי ברכבים, בהתאם לחיפושים עתידיים (`PickUpDT`), ומעבירה אליהם רכבים מסניפים אחרים באותה מדינה שבהם יש עודף רכבים. ההיגיון העסקי הוא להבטיח זמינות רכבים בסניפים עם ביקוש גבוה, לשפר את שביעות רצון הלקוחות, ולמנוע עלויות מיותרות שנובעות מרכבים שאינם מנוצלים. השימוש בפרוצדורה קריטי מכיוון שהיא מרכזת לוגיקה מורכבת, כולל חישובי ביקוש וזמינות, ומאפשרת ניהול דינמי וחכם של המשאבים, ללא צורך בעדכונים ידניים.

```
CREATE PROCEDURE AdjustBranchResourcesByFuturePickUp
    @TargetMonth INT,
    @TargetYear INT,
    @TargetCountry NVARCHAR(50)
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @BranchDemand TABLE (
        BranchID INT,
        Demand INT
    );

    INSERT INTO @BranchDemand
    SELECT
        S.CollectBranchID AS BranchID,
        COUNT(*) AS Demand
    FROM
        SEARCHES S
    JOIN
        BRANCHES B ON S.CollectBranchID = B.BranchID
    WHERE
        B.AdressCountry = @TargetCountry
        AND MONTH(S.PickUpDT) = @TargetMonth
        AND YEAR(S.PickUpDT) = @TargetYear
    GROUP BY
        S.CollectBranchID;

    DECLARE @BranchID INT, @Demand INT, @AvailableCars INT;
    DECLARE @FromBranchID INT, @LicensePlate NVARCHAR(6);

    DECLARE DemandCursor CURSOR FOR
    SELECT BranchID, Demand
    FROM @BranchDemand;

    OPEN DemandCursor;

    FETCH NEXT FROM DemandCursor INTO @BranchID, @Demand;
```




```
WHILE @@FETCH_STATUS = 0
BEGIN
    SELECT @AvailableCars = COUNT(*)
    FROM CARS
    WHERE BranchID = @BranchID;

    IF @Demand > @AvailableCars
    BEGIN
        DECLARE @CarsToTransfer INT = @Demand - @AvailableCars;

        DECLARE CarsCursor CURSOR FOR
        SELECT TOP (@CarsToTransfer) BranchID, LicensePlate
        FROM CARS
        WHERE BranchID IN (
            SELECT BranchID
            FROM BRANCHES
            WHERE AddressCountry = @TargetCountry AND BranchID <> @BranchID
        )
        AND LicensePlate NOT IN (
            SELECT SR.LicensePlate
            FROM ORDERS O
            JOIN SEARCH_RESULTS SR ON O.ResultID = SR.ResultID
        );

        OPEN CarsCursor;

        FETCH NEXT FROM CarsCursor INTO @FromBranchID, @LicensePlate;

        WHILE @@FETCH_STATUS = 0
        BEGIN
            UPDATE CARS
            SET BranchID = @BranchID
            WHERE LicensePlate = @LicensePlate;

            PRINT CONCAT('Transferred car with LicensePlate: ', @LicensePlate,
                ' from BranchID: ', @FromBranchID,
                ' to BranchID: ', @BranchID);

            FETCH NEXT FROM CarsCursor INTO @FromBranchID, @LicensePlate;
        END

        CLOSE CarsCursor;
        DEALLOCATE CarsCursor;
    END
ELSE
    BEGIN
        PRINT CONCAT('No cars need to be transferred to branch ', @BranchID);
    END
```



```
FETCH NEXT FROM DemandCursor INTO @BranchID, @Demand;
END
```

```
CLOSE DemandCursor;
DEALLOCATE DemandCursor;
END;
GO
```

שורת ביצוע:

```
EXEC AdjustBranchResourcesByFuturePickUp
@TargetMonth = 2,
@TargetYear = 2024,
@TargetCountry = 'Italy';
```

לפני הביצוע:

	BranchID	AdressCountry	AdressCity	TotalCars
1	126	Italy	Naples	1
2	131	Italy	Naples	0

הרצת הפרוצדורה:

100 % ▾

Messages

No cars need to be transferred to branch 69

No cars need to be transferred to branch 71

No cars need to be transferred to branch 75

No cars need to be transferred to branch 79

No cars need to be transferred to branch 82

No cars need to be transferred to branch 83

No cars need to be transferred to branch 84

No cars need to be transferred to branch 85

No cars need to be transferred to branch 90

No cars need to be transferred to branch 95

No cars need to be transferred to branch 102

No cars need to be transferred to branch 105

No cars need to be transferred to branch 106

No cars need to be transferred to branch 109

No cars need to be transferred to branch 110

No cars need to be transferred to branch 113

No cars need to be transferred to branch 114

No cars need to be transferred to branch 118

No cars need to be transferred to branch 119

No cars need to be transferred to branch 120

No cars need to be transferred to branch 121

No cars need to be transferred to branch 122

No cars need to be transferred to branch 123

No cars need to be transferred to branch 125

Transferred car with LicensePlate: 08W2ME from BranchID: 79 to BranchID: 126

No cars need to be transferred to branch 128

No cars need to be transferred to branch 129

Transferred car with LicensePlate: 08W2ME from BranchID: 126 to BranchID: 131

No cars need to be transferred to branch 132

לאחר הביצוע:

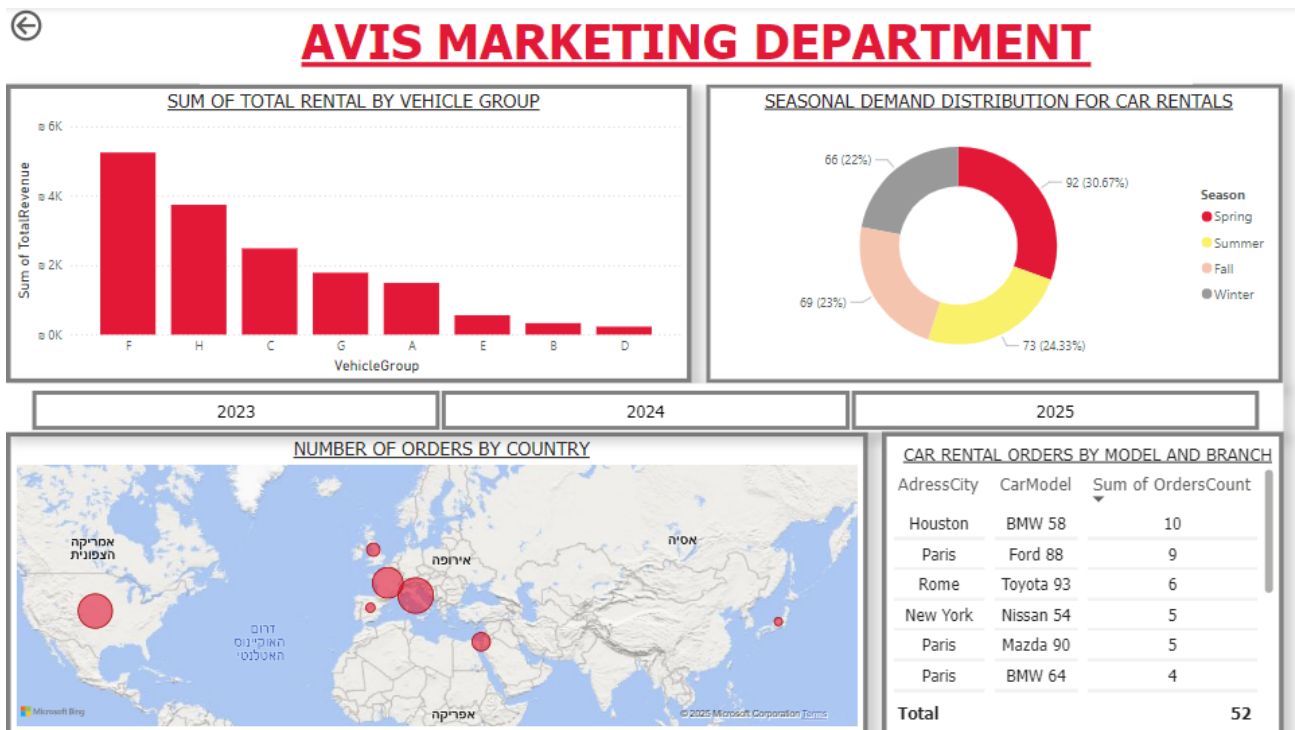
	BranchID	AdressCountry	AdressCity	TotalCars
1	126	Italy	Naples	1
2	131	Italy	Naples	1



כלים להצגת נתונים

דוח עסקי-למחלקת השיווק

נציג את הדוח העסקי שבנינו עבור מחלקת השיווק של חברת AVIS. הדוח נועד להעריך את הביצועים בשוק השכרת הרכב במהלך השנים האחרונות ולסייע למחלקת השיווק להגדיל את הביקוש להשכרת רכבים ולהגביר את הרווחים באמצעות קמפיינים שיווקיים ממומנים. הוא מציע כלים שיסייעו למקד את הפעילות השיווקית ולהתאים את האסטרטגיות לצרכים המשתנים של החברה, תוך זיהוי תחומים לשיפור והכוונה בקבלת החלטות מבוססות נתונים וטרנדים.



מרכיבי הדוח מפורטים כדלקמן:

מפת ביקוש לפי מדינות- המפה מציגה את מספר ההזמנות לפי מדינות. על ידי ניתוח מפת הביקוש, ניתן להבין באילו מדינות יש את הביקוש הגבוה ביותר להשכרת רכבים, ולהתמקד בהן עם קמפיינים שיווקיים ממומנים שיכולים להגדיל את נוכחות AVIS במדינות אלו.

סכום ההשכרות לפי קבוצות רכבים- בעזרת גרף עמודות נציג את סכום ההכנסות מכל קבוצת רכבים. הנתונים מסייעים למחלקת השיווק להבין אילו קבוצות רכבים מניבות את הרווחים הגבוהים ביותר. ניתוח נתונים זה מאפשר למקד את הקמפיינים השיווקיים בקבוצות רכבים פופולריות ולהתאים את האסטרטגיות השיווקיות לקבוצות רכבים המבוקשות ביותר, תוך שמירה על אופטימיזציה של ההכנסות.



ביקוש עונתי להשכרת רכבים- בעזרת תרשים עוגה נציג את הביקוש להשכרת רכבים לפי עונות השנה. ניתוח זה מאפשר להבין מתי יש עלייה או ירידה בביקוש ולתכנן קמפיינים שיווקיים שמתאימים לעונות השנה השונות (חורף, אביב, קיץ וסתיו).

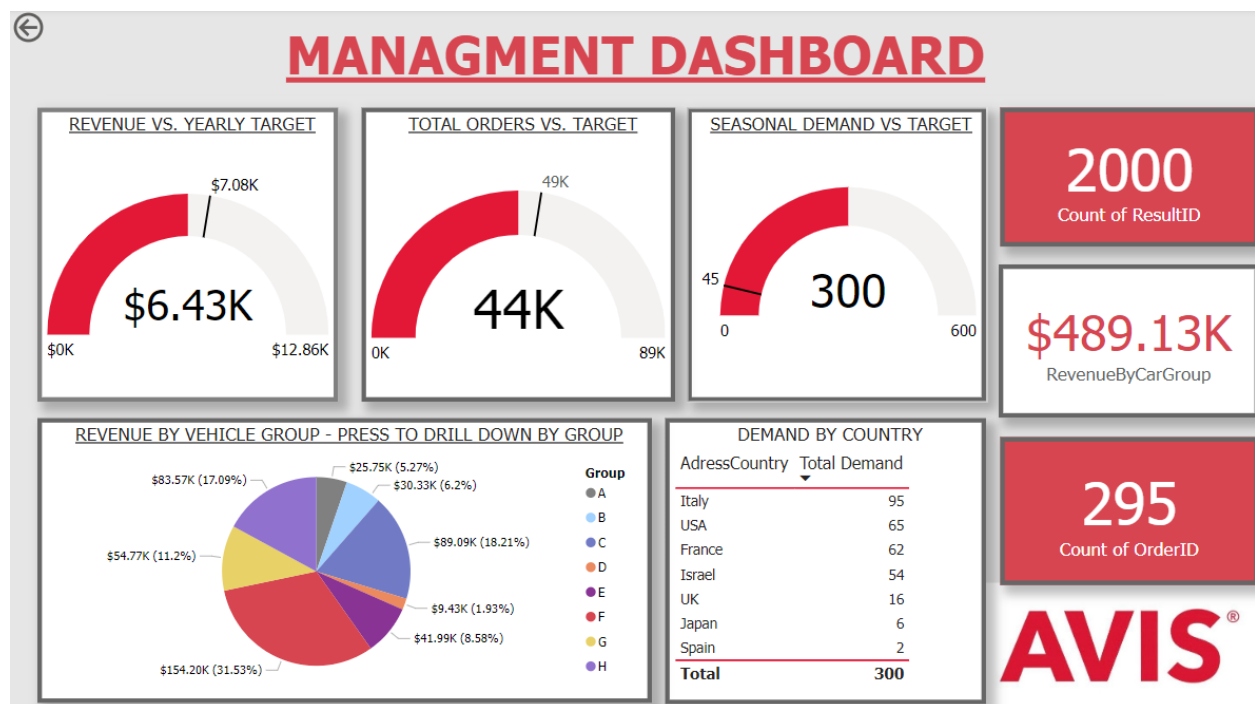
סך כל הזמנות הרכב לפי דגם רכב ועיר סניף- בעזרת טבלה נציג את סך כל ההזמנות לפי דגם רכב ועיר סניף. הטבלה מסייעת למחלקת השיווק לזהות את הדגמים הפופולריים ביותר בכל סניף ולבצע אופטימיזציה של קמפיינים שיווקיים ממומנים. באמצעות ניתוח נתונים אלו, ניתן להבין אילו דגמים מבוקשים ביותר בכל סניף גיאוגרפי ולהתאים את האסטרטגיות השיווקיות לקידום דגמים אלו באזורים עם ביקוש גבוה. בנוסף, הנתונים יכולים לשמש להכוונה טובה יותר של מלאי הרכבים והקצאת משאבים בסניפים השונים. באמצעות תובנות אלו, מחלקת השיווק יכולה ליצור קמפיינים ממוקדים, לייעל את מלאי הרכבים בסניפים ולהתאים את האסטרטגיות השיווקיות לצרכים המשתנים של החברה.



לוח המחוונים

נציג את לוח המחוונים שבנינו עבור הנהלת חברת השכרת רכבים. בלוח המחוונים נציג מידע ויזואלי ותמציתי הכולל את מדדי המפתח ונקודות רלוונטיות נוספות. בנוסף, ניתן למנהלי החברה אפשרות להתעמק בנתונים ברמת גרעיניות נמוכה יותר באמצעות כלי Drill Down. הדוח מספק מידע מקיף ובזמן אמת על ביצועי החברה, במטרה לאפשר קבלת החלטות מושכלות. מדדי המפתח של החברה:

- הכנסות
- כמות הזמנות
- ביקוש לפי סניפים ואזורים
- עונות עם עלייה/ירידה בביקוש להשכרות רכבים
- הכנסות לפי קבוצות רכבים



מרכיבי הדוח מפורטים כדלקמן:

נתונים בזמן אמת - בלוח המחוונים יש כרטיסיות המציגות נתונים בזמן אמת:

- כמות חיפושים שנעשו באתר בזמן אמת
- סך ההכנסות לפי קבוצות רכבים
- סך ההזמנות שבוצעו

מדדי כוח עמידה מול יעדים – במדדי הכוח ניתן לראות את מצב ביצועי החברה מול יעדי החברה בנושאים הבאים:

- הכנסות מהשכרות
- כמות הזמנות
- סך הביקוש להשכרות רכבים לפי עונה



מדדים אלה מאפשרים למנכ"ל:

- לעקוב אחר ההתקדמות מול היעדים שהוגדרו.
- לזהות בעיות פוטנציאליות בזמן אמת ולנקוט פעולות לתיקון.
- לקבל תמונת מצב מעודכנת על ביצועי החברה ולשפר את תהליך קבלת החלטות העסקיות.
- למקד את המאמצים בתחומים שדורשים שיפור ולהגביר את הרווחיות והביצועים של החברה.

ביקוש לפי סניפים ואזורים – באמצעות טבלה זו ניתן לראות את הביקוש להשכרת רכבים לפי סניפים ואזורים גיאוגרפיים. מנהלי החברה יוכלו לבחון זיהוי סניפים ואזורים בעלי ביצועים טובים או נמוכים מהמצופה ולקבל החלטות על התאמת מלאי, פרסום, או שיווק ממוקד באזורים מסוימים.

הכנסות לפי קבוצות רכבים – גרף העוגה מציג את הפילוח של ההכנסות לפי קבוצות רכבים. כמו כן ניתן לראות כי השימוש ב-Drill Down מאפשר להתעמק בפלח מסוים (לדוגמה, קבוצה מסוימת של רכבים) ולבחון פילוח מפורט לפי אזורים גיאוגרפיים, סניפים וכדומה. מנהלי החברה יוכלו לזהות קבוצות רכבים עם פוטנציאל שיווקי גבוה ולקבל החלטות על קידום או השקעה בקבוצות רכבים מסוימות בהתאם לביצועים.



אופטימיזציה של שאלות באמצעות Generative AI

קישור לצ'אט

1. שאלתה המשתמשת בפונקציות חלון ומבצעת חלוקה בהתאם לעונות השנה. בשאלתה שודרגה היעילות על ידי צמצום השימוש ב CASE ו DATEPART - והחלפתם בשדות מחושבים בתוך JOIN ישיר עם טבלת SEARCHES. נוסף לכך, הוספנו אינדקסים על שדות שימושיים כמו BranchID, ResultID, LicensePlate - לשיפור זמני החיפוש והחיבורים.

```
WITH SeasonCTE AS (
    SELECT
        S.PickUpDT,
        CASE
            WHEN DATEPART(QUARTER, S.PickUpDT) = 1 THEN 'Winter'
            WHEN DATEPART(QUARTER, S.PickUpDT) = 2 THEN 'Spring'
            WHEN DATEPART(QUARTER, S.PickUpDT) = 3 THEN 'Summer'
            WHEN DATEPART(QUARTER, S.PickUpDT) = 4 THEN 'Fall'
        END AS SeasonName,
        S.IPAddress,
        S.SearchDT
    FROM
        SEARCHES S
)
SELECT
    CM.[Group] AS VehicleGroup,
    SC.SeasonName,
    COUNT(O.OrderID) AS TotalOrders,
    SUM(SR.CarPrice) AS TotalRevenue,
    RANK() OVER (PARTITION BY SC.SeasonName ORDER BY SUM(SR.CarPrice) DESC) AS
RevenueRank,
    NTILE(4) OVER (PARTITION BY SC.SeasonName ORDER BY COUNT(O.OrderID) DESC) AS
OrderQuartile
FROM
    ORDERS O
JOIN
    SEARCH_RESULTS SR ON O.ResultID = SR.ResultID
JOIN
    RETRIVALS R ON SR.ResultID = R.ResultID
JOIN
    SeasonCTE SC ON SC.IPAddress = R.IPAddress AND SC.SearchDT = R.SearchDT
JOIN
    CARS C ON SR.LicensePlate = C.LicensePlate
JOIN
    CARMODELS CM ON C.CarModel = CM.CarModel
JOIN
    BRANCHES BR ON C.BranchID = BR.BranchID
WHERE
    BR.AddressCountry IN ('France', 'Germany', 'Spain', 'Italy', 'United Kingdom')
```



GROUP BY

CM.[Group],
SC.SeasonName

ORDER BY

SC.SeasonName, RevenueRank;

אינדקסים שנוספו:

CREATE INDEX IDX_SEARCH_RESULTS_ResultID
ON SEARCH_RESULTS (ResultID);

CREATE INDEX IDX_RETRIEVALS_ResultID
ON RETRIVALS (ResultID);

CREATE INDEX IDX_CARS_LicensePlate
ON CARS (LicensePlate);

CREATE INDEX IDX_BRANCHES_BranchID
ON BRANCHES (BranchID);

CREATE INDEX IDX_BRANCHES_AdressCountry
ON BRANCHES (AdressCountry);



2. שאילתת WITH שמנתחת את פעילות החיפושים וההזמנות העתידיות באתר. השינויים שביצענו כוללים את החלפת ה-UNION ב-FULL OUTER JOIN, חישוב ה-NTILE פעם אחת על הנתונים הממוזגים, ושימוש ב-INNER JOIN במקום ב-LEFT JOIN בין טבלאות. שיפורים אלו צמצמו כפילויות נתונים, הפחיתו את זמן הריצה על ידי הפחתת חישובים מיותרים, והפכו את השאילתה ליעילה יותר מבחינת ביצועים.

WITH

```
SearchActivity AS (
    SELECT
        S.CollectBranchID AS BranchID,
        COUNT(*) AS TotalFutureSearches
    FROM
        SEARCHES S
    WHERE
        S.PickUpDT >= DATEADD(MONTH, 1, '2024-01-01')
    GROUP BY
        S.CollectBranchID
),
OrderActivity AS (
    SELECT
        B.BranchID,
        COUNT(O.OrderID) AS TotalFutureOrders
    FROM
        BRANCHES B
    INNER JOIN
        SEARCHES S ON B.BranchID = S.CollectBranchID
    INNER JOIN
        RETRIVALS R ON S.IpAddress = R.IpAddress AND S.SearchDT = R.SearchDT
    INNER JOIN
        SEARCH_RESULTS SR ON R.ResultID = SR.ResultID
    INNER JOIN
        ORDERS O ON SR.ResultID = O.ResultID
    WHERE
        S.PickUpDT >= DATEADD(MONTH, 1, '2024-01-01')
    GROUP BY
        B.BranchID
),
RankedBranches AS (
    SELECT
        COALESCE(SA.BranchID, OA.BranchID) AS BranchID,
        COALESCE(SA.TotalFutureSearches, 0) AS TotalFutureSearches,
        COALESCE(OA.TotalFutureOrders, 0) AS TotalFutureOrders
    FROM
        SearchActivity SA
    FULL OUTER JOIN
        OrderActivity OA ON SA.BranchID = OA.BranchID
)
SELECT
    RB.BranchID,
```



```
B.AddressCity,  
B.AddressCountry,  
RB.TotalFutureSearches,  
RB.TotalFutureOrders,  
CEILING((RB.TotalFutureSearches + RB.TotalFutureOrders) * 1.2) AS RequiredVehicles,  
NTILE(5) OVER (ORDER BY RB.TotalFutureSearches DESC) AS PopularityRank  
FROM  
RankedBranches RB  
JOIN  
BRANCHES B ON RB.BranchID = B.BranchID  
ORDER BY  
PopularityRank ASC, RB.TotalFutureSearches DESC;
```



יישום כלים נוספים בלימוד עצמי

TRY...CATCH

הפרוצדורה AddCreditCard נועדה להתמודד עם בעיות בהכנסת פרטי כרטיסי אשראי בצורה חכמה וידידותית. באמצעות מנגנון TRY...CATCH, היא בודקת את תקינות מספר הכרטיס, CVV ותוקף הכרטיס, ומחזירה למשתמש הודעה ברורה וממוקדת על מקור התקלה, מה שמספר את חוויית המשתמש ומסייע לו לתקן את הבעיה במהירות. בנוסף, כל ניסיון כושל נרשם בטבלת לוג ייעודית (InvalidCreditCardsLog), הכוללת את פרטי המשתמש, הכרטיס, והסיבה לכישלון. גישה זו מאפשרת לי, כחברה, לשמור על קשר עם המשתמש שלא הצליח להשלים את התשלום, ולנסות לסייע לו בדרכים אחרות, בין אם באמצעות פנייה ישירה ובין אם בהצעת שיטות תשלום חלופיות. כך, אני לא רק מונע אובדן לקוח, אלא גם משפר את שביעות רצון המשתמש ואת הסיכוי להשלמת העסקה.

-- יצירת טבלת לוג למעקב אחרי כרטיסים לא תקינים --

```
CREATE TABLE InvalidCreditCardsLog (
  LogID INT IDENTITY(1,1) PRIMARY KEY,
  UserID VARCHAR(9) NOT NULL,
  AttemptedCardNum VARCHAR(16),
  AttemptedCVV CHAR(3),
  AttemptedExpiration DATE,
  ErrorMessage NVARCHAR(4000),
  AttemptDate DATETIME DEFAULT GETDATE()
);
```

-- יצירת פרוצדורה עם פירוט על שגיאות בכרטיס אשראי --

```
CREATE PROCEDURE AddCreditCard
  @CardNum VARCHAR(16),
  @Expiration DATE,
  @CVV CHAR(3),
  @UserID VARCHAR(9)
AS
BEGIN
  BEGIN TRY
    -- בדיקת תקינות של מספר הכרטיס --
    IF LEN(@CardNum) <> 16 OR @CardNum NOT LIKE '[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]'
    BEGIN
      THROW 51000, 'Invalid credit card number.', 1;
    END

    -- בדיקת תקינות של CVV --
    IF LEN(@CVV) <> 3 OR @CVV NOT LIKE '[0-9][0-9][0-9]'
    BEGIN
      THROW 51001, 'Invalid CVV.', 1;
    END
  END TRY
  BEGIN CATCH
    -- יצירת רשומה בטבלת הלוג --
    INSERT INTO InvalidCreditCardsLog (
      UserID,
      AttemptedCardNum,
      AttemptedCVV,
      AttemptedExpiration,
      ErrorMessage
    ) VALUES (
      @UserID,
      @CardNum,
      @CVV,
      @Expiration,
      ERROR_MESSAGE()
    );
  END CATCH
END
```



```
-- בדיקת תוקף הכרטיס
IF @Expiration <= GETDATE()
BEGIN
    THROW 51002, 'Credit card expiration date has passed.', 1;
END

-- CREDIT_CARDS ניסיון להכניס את הכרטיס לטבלת
INSERT INTO CREDIT_CARDS (CardNum, Expiration, CVV, UserID)
VALUES (@CardNum, @Expiration, @CVV, @UserID);

PRINT 'Credit card added successfully.';
END TRY
BEGIN CATCH
    -- טיפול בשגיאה והוספת הרשומה לטבלת הלוג
    DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();

    -- שמירת המידע של הניסיון בטבלת הלוג
    INSERT INTO InvalidCreditCardsLog (UserID, AttemptedCardNum, AttemptedCVV,
    AttemptedExpiration, ErrorMessage)
    VALUES (@UserID, @CardNum, @CVV, @Expiration, @ErrorMessage);

    -- הדפסת הודעה למשתמש
    PRINT 'Failed to add credit card. Error logged.';
    PRINT 'Error Message: ' + @ErrorMessage;
END CATCH
END;
GO
```

דוגמא לטבלת הלוג

	LogID	UserID	AttemptedCardNum	AttemptedCVV	AttemptedExpiration	ErrorMessage	AttemptDate
1	1	987654321	1234567890123456	12	2025-12-31	Violation of PRIMARY KEY constraint 'PK__CREDIT_...	2025-01-23 13:43:41.457
2	2	567891234	1234567890123456	123	2022-01-01	Violation of PRIMARY KEY constraint 'PK__CREDIT_...	2025-01-23 13:46:14.410
3	3	987654321	1234567890123456	12	2025-12-31	Invalid CVV.	2025-01-23 13:48:15.123
4	4	345678901	12345678901234	123	2025-12-31	Invalid credit card number.	2025-01-23 15:04:06.933
5	5	345678901	12345678901234	123	2025-12-31	Invalid credit card number.	2025-01-23 15:04:12.517
6	6	567891234	1234567890123456	123	2022-01-01	Credit card expiration date has passed.	2025-01-23 15:04:23.243

דוגמא לניסיון הכנסת כרטיס עם CVV שגוי

Messages

```
(1 row affected)
Failed to add credit card. Error logged.
Error Message: Invalid CVV.
```



Temporary Tables

השאלתה מחברת נתונים ממספר טבלאות במערכת כדי לחשב את סך ההכנסות לכל סניף, בהתבסס על חיפושי רכבים והזמנות שבוצעו. השימוש בטבלה זמנית מאפשר חישוב מדויק של מחירי רכבים לכל חיפוש, מבלי להשפיע על ביצועי הטבלאות הראשיות. גישה זו מאפשרת לבצע את החישוב בצורה ממודלת ויעילה, ומסייעת בקבלת החלטות עסקיות לגבי ביצועי הסניפים על בסיס נתונים מדויקים.

```
IF OBJECT_ID('tempdb..#TempBranchRevenue') IS NOT NULL
    DROP TABLE #TempBranchRevenue;
```

```
CREATE TABLE #TempBranchRevenue (
    BranchID INT,
    TotalRevenue MONEY
);
```

-- שלב החיבור והשאיבה

```
WITH UniqueRetrivals AS (
    SELECT
        r.ResultID,
        r.IpAddress,
        r.SearchDT,
        ROW_NUMBER() OVER (PARTITION BY r.ResultID ORDER BY s.SearchDT DESC) AS
        RowNum
    FROM RETRIVALS r
    INNER JOIN SEARCHES s
        ON r.IpAddress = s.IpAddress AND r.SearchDT = s.SearchDT
)
```

```
INSERT INTO #TempBranchRevenue (BranchID, TotalRevenue)
SELECT
    s.CollectBranchID AS BranchID, -- סניף האיסוף
    SUM(sr.CarPrice) AS TotalRevenue
FROM UniqueRetrivals ur
INNER JOIN SEARCH_RESULTS sr
    ON ur.ResultID = sr.ResultID
INNER JOIN ORDERS o
    ON o.ResultID = sr.ResultID
INNER JOIN SEARCHES s
    ON ur.IpAddress = s.IpAddress AND ur.SearchDT = s.SearchDT
WHERE ur.RowNum = 1
GROUP BY s.CollectBranchID;
```

-- שלב הצגת התוצאה

```
SELECT
    b.BranchID,
    b.AdressCity,
    b.AdressStreet,
    b.Phone,
```



```
t.TotalRevenue
FROM #TempBranchRevenue t
INNER JOIN BRANCHES b
  ON t.BranchID = b.BranchID
ORDER BY t.TotalRevenue DESC;
```

```
-- ניקוי הטבלה הזמנית --
DROP TABLE #TempBranchRevenue;
```

הפלט שהתקבל:

	BranchID	AdressCity	AdressStreet	Phone	TotalRevenue
1	43	Rome	Pine Ave	1443032398	860.75
2	17	Paris	Maple Rd	7715245230	728.25
3	105	Rome	Pine Ave	8074708616	673.20
4	38	Paris	Maple Rd	7724807060	595.98
5	58	New York	Maple Rd	9518014914	570.63
6	28	Houston	Pine Ave	2690933660	521.63
7	18	New York	Pine Ave	6018631747	508.84
8	67	New York	Elm St	9053479293	508.43
9	70	Paris	Elm St	3071412465	507.02
10	39	Tel Aviv	Maple Rd	7916775968	500.80
11	69	Rome	Main St	4178258885	475.63
12	79	Rome	Main St	6618163737	468.33